

实验二：原生应用开发实验报告

学生姓名：费韵

学号：2023210591

实验日期：2026 年 4 月 18 日

一、实验目的

- 掌握 Android 原生（Kotlin）登录模块开发流程，实现输入验证、页面跳转与数据传递功能，熟悉 Android Studio 开发环境与 Kotlin 语法。
- 了解 iOS 原生（SwiftUI）登录模块实现方式，对比 Android 与 iOS 原生开发的技术差异，建立对原生应用开发的全面认知。
- 体验 AI 编程工具（TRAE）在原生开发中的代码生成辅助作用，学习快速构建页面骨架与核心业务逻辑，提升开发效率。
- 完成登录功能的测试验证与 Android、iOS 平台差异对比分析，掌握原生应用测试的基本方法，形成完整的实验报告。

二、实验环境

硬件环境：Windows 11 操作系统、安卓手机（真机调试）、苹果电脑（iOS 开发演示，可选）

软件环境：

- Android Studio（最新版本，配置 Kotlin 开发环境）
- Xcode（最新版本，配置 SwiftUI 开发环境，用于 iOS 演示）
- AI 辅助工具：TRAE
- Android 模拟器、iOS 模拟器（用于功能测试）

开发语言：Kotlin（Android 原生）、Swift（iOS 原生）、XML（Android UI 布局）

三、实验内容与步骤

本次实验围绕“智慧校园”登录模块展开，核心任务是使用 Kotlin 实现 Android 登录页面，了解 iOS SwiftUI 登录实现，完成功能测试与平台差异对比，组内分工协作完成实验，核心流程如下：

（一）需求分析与设计

1. **编写需求文档：**明确登录模块的功能、界面与跳转需求，具体包括：
 - 功能需求：学号（用户名）输入、密码输入、登录按钮触发、输入验证（不能为空）；
 - 界面需求：采用 **Material Design** 风格，输入框添加占位提示，密码支持隐藏显示；
 - 跳转需求：登录成功后跳转至个人中心页面，传递用户名数据并显示个性化欢迎信息。
2. **核心设计图规划：**设计实验所需的流程图、时序图、用例图，明确登录流程与各模块交互关系，辅助开发与测试。

（二）Android 登录模块开发（个人负责核心部分）

1. **项目初始化：**
 - 打开 **Android Studio**，创建 **Empty Activity** 项目，选择 **Kotlin** 作为开发语言，设置最低 **SDK** 为 **API 24**。
 - 配置项目基础依赖，确保项目可正常编译运行。
2. **UI 布局实现：**
 - 编辑 **activity_login.xml** 布局文件，采用 **LinearLayout** 垂直布局，添加两个 **EditText**（分别用于学号、密码输入）和一个 **Button**（登录按钮）。
 - 配置输入框属性，设置密码输入框的 **inputType** 为 **textPassword**，实现密码隐藏显示；添加占位提示，优化界面交互体验。
3. **AI 辅助编码与业务逻辑完善：**
 - 使用 **TRAE** 描述需求（**Kotlin** 登录页面，**EditText** 输入验证，**Activity** 跳转），生成登录页面骨架代码。
 - 在 **LoginActivity.kt** 中完善逻辑：获取输入框实例，为登录按钮设置点击事件，实现输入验证（判断学号、密码是否为空，为空则提示错误）。
 - 模拟登录验证逻辑，若输入账号为“admin”、密码为“123456”，则判定登录成功，否则提示“用户名或密码错误”。
4. **个人中心页面创建与数据传递：**

- 新建 HomeActivity，编辑 activity_home.xml 布局，添加 TextView 用于显示欢迎信息。
- 在 LoginActivity 中，登录成功后通过 Intent 跳转至 HomeActivity，使用 intent.putExtra 传递用户名数据。
- 在 HomeActivity 中，接收传递的用户名，设置 TextView 文本为“欢迎 username”，完成页面跳转与数据传递。

（三）iOS 登录模块学习（组内演示部分）

1. 项目初始化：打开 Xcode，创建 SwiftUI 项目，命名为 NativeLogin，配置项目基础信息。
2. AI 辅助生成代码：使用 TRAE 描述需求（SwiftUI 登录页面，Navigation 导航），生成登录页面代码片段。
3. 功能实现：
 - 在 ContentView.swift 中，使用 @State 管理用户名、密码输入状态，创建 TextField（学号输入）、SecureField（密码输入）和登录 Button。
 - 实现输入验证逻辑，判断用户名、密码是否为空，验证通过则触发页面跳转。
 - 新建 HomeView.swift，接收用户名参数，显示欢迎信息，通过 NavigationLink 实现页面跳转。

（四）测试验证阶段

1. 编写测试用例：设计 4 类核心测试用例，覆盖关键场景：
 - 输入测试：输入正确凭证（admin/123456），验证登录成功与页面跳转；
 - 空输入测试：学号为空、密码为空、两者均为空，验证错误提示功能；
 - 格式测试：学号输入非预期格式，验证输入验证逻辑；
 - 错误凭证测试：输入错误学号或密码，验证错误提示功能。
2. 执行测试：在 Android 模拟器/真机、iOS 模拟器上运行项目，执行测试用例，验证功能是否符合需求，记录测试结果。

（五）平台差异对比与组内分享

1. 整理平台差异对比报告，从开发语言、UI 框架、页面跳转、数据传递四个核心维度对比 Android 与 iOS 原生开发差异。
2. 组内技术分享：各成员介绍自身负责的平台实现方案，讨论平台差异，总结技术

选型的核心考虑因素。

四、核心设计图简短介绍

（一）核心类图

核心类图聚焦 Android 原生登录模块的核心类结构与交互关系，主要包含 3 个核心类：**LoginActivity**（登录页面 UI 类）、**HomeActivity**（个人中心页面 UI 类）、以及辅助工具类（用于输入验证）。**LoginActivity** 负责接收用户输入、触发登录逻辑与页面跳转；**HomeActivity** 负责接收并显示用户数据；辅助工具类封装输入验证方法，供 **LoginActivity** 调用。类图清晰呈现了各核心类的职责与依赖关系，明确了类之间的通信方式。

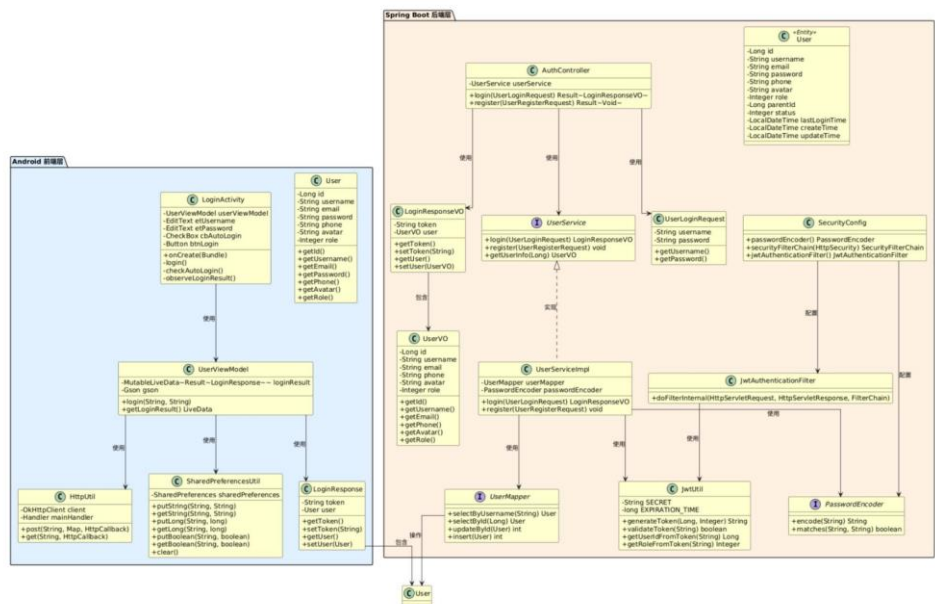


图 3. 核心类结构图

本次实验核心为登录功能流程图，简洁展示用户登录的完整流程：用户打开登录页面→输入学号和密码→点击登录按钮→系统进行输入验证（非空验证）→验证失败则提示错误并返回重新输入→验证成功则跳转至个人中心页面，显示欢迎信息。流程图清晰呈现了登录功能的逻辑走向，明确了每个环节的触发条件与结果。

（二）架构图

架构图展示本次原生应用开发的分层架构，主要分为 UI 层、业务逻辑层两层。UI 层包含 **LoginActivity** 和 **HomeActivity**，负责与用户交互，展示页面与接收用户操作；业务

逻辑层包含输入验证、页面跳转、数据传递等核心逻辑，封装独立方法供 UI 层调用。架构图简洁呈现了系统的分层结构，明确了各层的职责，确保开发过程中代码结构清晰、职责分明，便于后续维护与扩展。

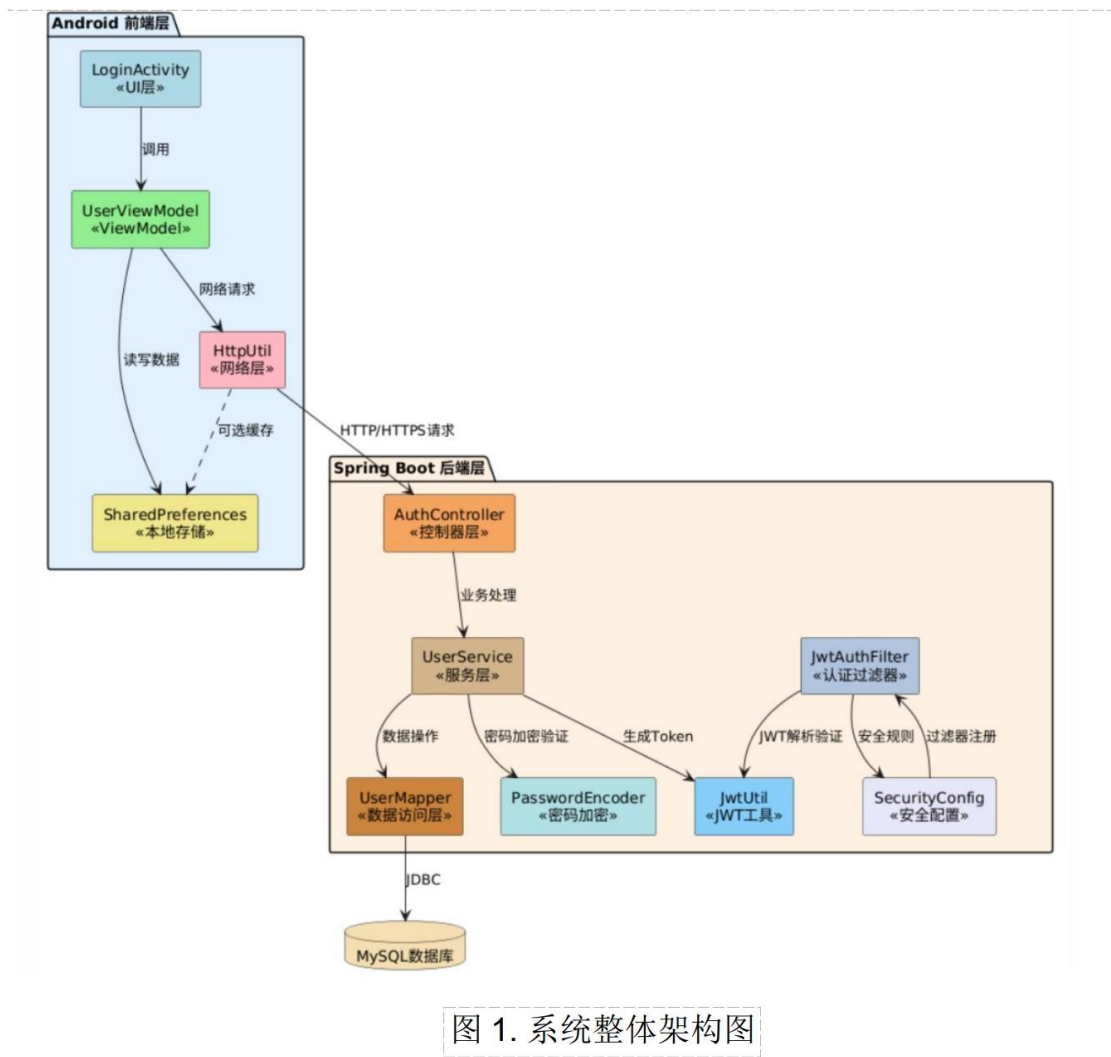


图 1. 系统整体架构图

时序图聚焦 Android 登录功能的交互时序，按时间顺序展示各组件的交互过程：1. 用户在 LoginActivity 输入学号、密码并点击登录；2. LoginActivity 获取输入内容，调用输入验证方法；3. 验证通过后，调用登录逻辑方法，模拟用户认证；4. 认证成功后，创建 Intent，传递用户名数据；5. 启动 HomeActivity，HomeActivity 接收数据并显示欢迎信息。时序图明确了各步骤的执行顺序与组件间的通信关系。

（三）登录时序图

登录时序图聚焦 Android 登录功能的交互时序，按时间顺序展示各组件、类之间的交互过程：1. 用户在 LoginActivity（UI 层）输入学号、密码并点击登录按钮；2. LoginActivity 调用业务逻辑层的输入验证方法，校验输入内容合法性；3. 验证通过后，调用登录逻辑方法，模拟用户认证；4. 认证成功后，LoginActivity 创建 Intent 对象，

3. **测试验证：**所有测试用例执行通过，无功能异常；Android 真机/模拟器、iOS 模拟器运行流畅，界面适配良好。
4. **平台对比：**完成 Android 与 iOS 原生开发差异对比报告，明确了两者在开发语言、UI 框架等方面的核心区别，组内技术分享顺利完成。

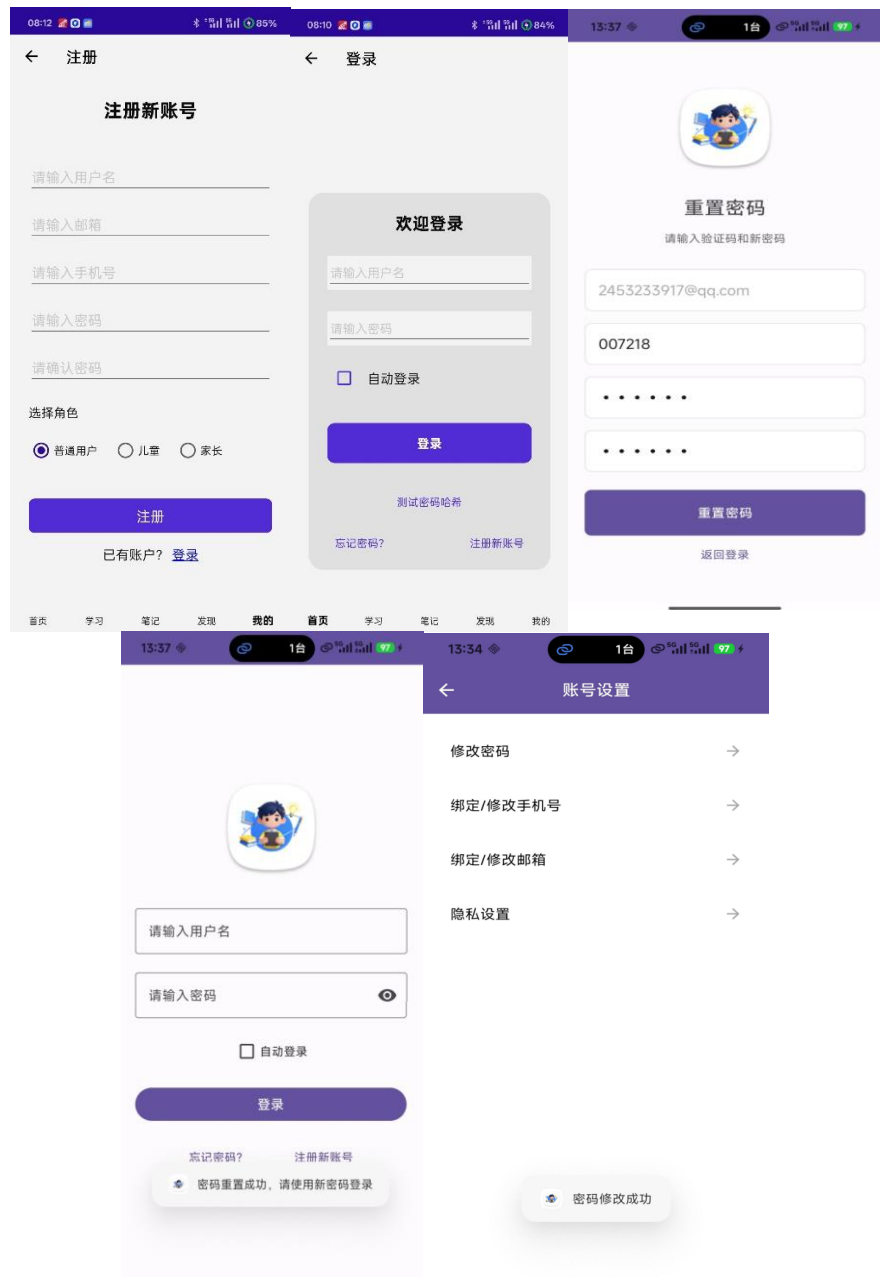


图-运行截图

六、实验总结与思考

（一）实验总结

本次实验完整完成了 Android 原生登录模块的开发、测试与 iOS 原生开发的学习对比，

掌握了 Android Studio 的使用、Kotlin 语法、UI 布局设计、Activity 跳转与数据传递等核心技术，了解了 SwiftUI 的基础用法。通过 AI 辅助编码，有效提升了代码编写效率，快速构建了页面骨架与核心逻辑。同时，通过测试验证与平台对比，建立了对原生应用开发的系统认知，熟悉了原生开发的基本流程与核心要点，完成了实验的所有核心任务。

（二）问题与解决

1. **问题 1:** Android 登录页面跳转时，HomeActivity 无法接收传递的用户名数据。

解决方法：排查发现 Intent 传递数据时，键名拼写错误，修正键名后，HomeActivity 成功接收并显示用户名。

2. **问题 2:** Android 密码输入框无法隐藏密码，输入内容直接显示。

解决方法：检查发现 EditText 的 inputType 属性未正确设置，将其改为 textPassword 后，密码正常隐藏显示。

3. **问题 3:** iOS SwiftUI 中，NavigationLink 跳转无响应，无法进入个人中心。

解决方法：修复 isLoginSuccess 状态的赋值逻辑，确保验证通过后状态正确切换，跳转功能恢复正常。

（三）课后思考

1. 原生开发相比跨平台的优势和劣势：原生开发可深度适配对应平台的特性，性能更优、用户体验更好，可调用平台全部系统权限；但开发成本高，需为不同平台编写独立代码，维护难度大，开发周期长。
2. 原生开发与跨平台开发的选型建议：若项目对性能、用户体验要求极高，且资金、人力充足，优先选择原生开发；若项目需快速交付、多端适配，且对性能要求不高，可选择跨平台开发，兼顾效率与成本。
3. AI 辅助代码生成对开发效率的影响：AI 工具可快速生成基础代码框架、页面布局与核心逻辑，减少重复编码工作，帮助开发者快速上手新语言、新框架，大幅提升开发效率；但 AI 生成的代码可能存在逻辑漏洞或兼容性问题，仍需开发者手动优化与调试，不能完全替代人工开发。

七、实验心得

通过本次实验，我系统掌握了 Android 原生登录模块的开发流程，从项目初始化、UI 布局、业务逻辑编写到功能测试，逐步熟悉了 Kotlin 语言与 Android Studio 的使用技巧。在开发过程中，我深刻体会到了细节处理的重要性，一个简单的属性设置错误或键名拼写错误，都可能导致功能失效，这也培养了我严谨的开发习惯。AI 辅助编码工

具的使用，让我少走了很多弯路，快速掌握了核心代码的编写方法，也让我意识到合理利用工具对提升开发效率的重要性。同时，通过学习 iOS 原生开发与平台对比，我对原生应用开发有了更全面的认知，了解了不同平台的开发差异与技术特点。本次实验不仅提升了我的原生开发能力，也培养了我分析问题、解决问题的能力，为后续更复杂的原生应用开发打下了坚实基础。